

Kubernetes and Cloud Native Associate - KCNA

Kubernetes Fundamentals

Kubernetes Architecture

Welcome to this lesson on Kubernetes Architecture. Before setting up a Kubernetes cluster, it's essential to understand the fundamental components and terminology used throughout the configuration process. This knowledge will help you design a robust, scalable, and highly available system.

Core Concepts

In Kubernetes terminology, a **node** refers to a machine—either physical or virtual—that runs the Kubernetes software. Nodes, formerly known as minions, are the worker machines that run containerized applications. Ensuring high availability and optimal performance requires deploying multiple nodes in a clustered configuration. A cluster is a group of nodes that work together: if one node fails, the remaining nodes continue to run your applications.

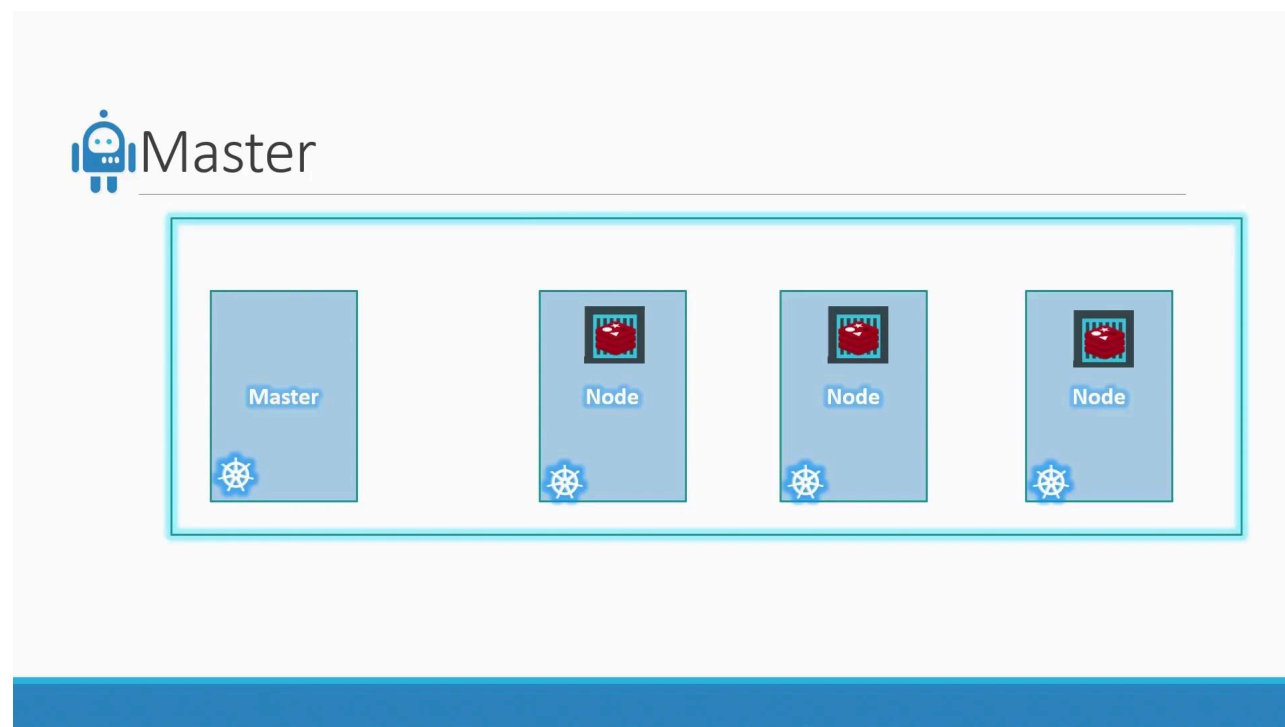


Note

The master node is responsible for overseeing and managing the cluster by monitoring node statuses and coordinating container deployments.

Role of the Master Node

The master node plays a crucial role in managing the Kubernetes cluster. It keeps track of all worker nodes and their statuses, orchestrates deployments, and ensures that the desired state of the cluster is maintained. The following diagram illustrates a typical Kubernetes cluster architecture with one master node and multiple worker nodes:



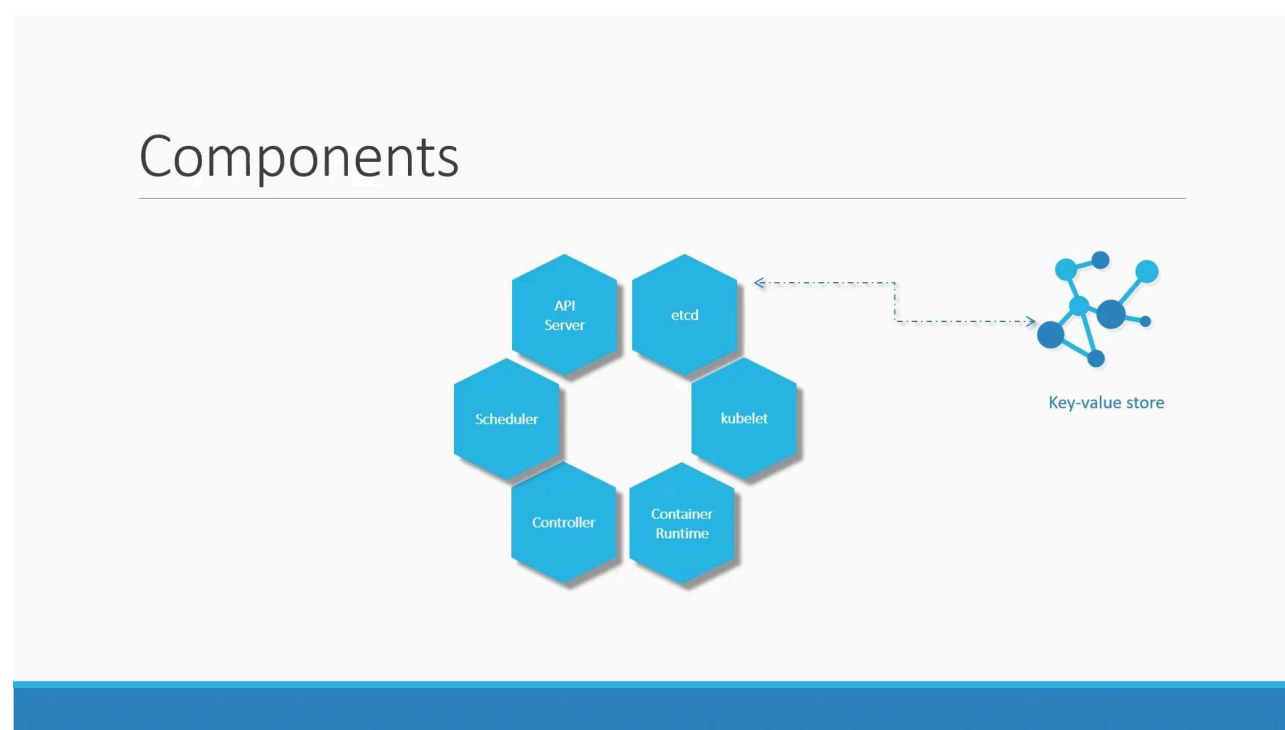
Key Kubernetes Components

When installing Kubernetes, several essential components are set up to form the control plane and enable seamless container orchestration:

- **API Server:** Serves as the front end for Kubernetes, mediating communication between users, the command line interface, and the cluster.
- **etcd:** A distributed key-value store that holds all the crucial data for managing the cluster. In multi-master environments, etcd ensures data consistency by replicating information and implementing lock mechanisms.
- **Scheduler:** Allocates newly created container tasks to suitable worker nodes based on resource availability.

- **Controllers:** Act as the orchestration brain by monitoring cluster events and replacing failed containers or nodes.
- **Container Runtime:** The underlying software responsible for running containers on a node. While Docker is popular, alternatives like containerd or CRI-O are available.
- **kubelet:** An agent on each worker node that communicates with the master node to ensure that containers are operational and healthy.

The diagram below provides a visual representation of these Kubernetes components:



Understanding how these components interact clarifies the distinction between master and worker nodes. Worker nodes host the containers and include elements like the container runtime (e.g., Docker) and the kubelet agent, while the master node comprises components such as the kube-apiserver, controller manager, scheduler, and an etcd-based key-value store.

Master vs Worker Nodes



Managing your Cluster with kubectl

An indispensable tool for interacting with your Kubernetes cluster is the `kubectl` command-line utility. With `kubectl`, you can deploy and manage applications, retrieve vital cluster information, and check node statuses. Here are some foundational `kubectl` commands:

- **kubectl run:** Deploys an application on the cluster.
- **kubectl cluster-info:** Displays detailed information about the cluster.
- **kubectl get nodes:** Lists all nodes in the cluster.

Below is an example demonstrating these commands:

```
kubectl run hello-minikube
kubectl cluster-info
kubectl get nodes
```

These commands serve as the foundation for efficient cluster management. As you progress through the lesson, you will learn more advanced `kubectl` functionalities and additional Kubernetes concepts.

That concludes this lesson on Kubernetes Architecture. Happy clustering!



Watch Video



Beta



● Search docs...



Kubernetes and Cloud Native Associate - KCNA ▾